WRCF (Workforce Readiness Capability Framework)

Whizard operates on a Work-Object-Centric Workforce Readiness Capability Framework (WRCF) structured as:

Sector
Industry
→ Functional Group
→ Primary Work Object (PWO)
→ Secondary Work Object (SWO)
→ Capability
→ Proficiency
→ Skill
→ Task
→ Control Point
→ Evidence
→ Evaluator

As per WRCF

- Capability Instances belong to the Industry.
- Competence belongs to the Role.
- People are assessed against Role Competence.
- Role competence is achieved through verified Capability Instances.
- A Capability Instance = SWO + Capability + Proficiency.

WRCF standardizes:

- Industry definitions
- Role definitions
- Functional groups
- Capability structures
- Skill evidences
- Certification pathways

Whizard is not a job portal. It is a capability-governed talent incubation and industry-integration platform.

Definitions, Acronyms, and Abbreviations

Workforce Readiness Capability Framework — A structured model defining roles, capabilities, skills, control points, and evidence aligned to industry standards.

Sector: A broad economic grouping of industries; provides macro-level context; not an organization or curriculum.

1 Industry

Definition:
A broad economic domain in which organizations operate, defined by a common production logic, regulatory structure, and value chain.

Purpose in WRCF:
Provides the **contextual ecosystem** within which capability models are built.

Examples:

Thermal Power Generation

Retail Banking

IT Software Development

Key Principle:
Industry defines *what kind of work exists*, not who performs it.

2 Company

Definition:
A specific legal entity operating within an industry that has its own asset base, processes, standards, and operating philosophy.

Purpose in WRCF:
Company is where **industry framework gets customized** (SOPs, equipment make, compliance rules).

Key Distinction:

Industry = generic capability model

Company = contextual implementation of that model

Example:

Industry: Thermal Power

Company: NTPC / Tata Power / State GENCO

3 Functional Group

Definition:
A major operational or support domain within an industry where related systems and processes are grouped.

Purpose in WRCF:
Organizes work into manageable operational clusters.

Examples (Thermal Power):

Coal Handling Plant (CHP)

Boiler

Turbine

Generator

Electrical Systems

C&I

Ash Handling

Key Insight:
Functional Group = where work happens.

4 Primary Work Object (PWO)

Definition:
A major system, process cluster, or operational domain within a functional group that delivers a distinct operational purpose.

Purpose in WRCF:
Defines the **competency boundary** for capability modeling.

Examples (CHP):

Coal Receiving System

Conveying Systems

Crushing & Sizing

Stacking & Reclaiming

Dust, Environment & Safety

Key Insight:
PWO = What major system must be mastered.

5 Secondary Work Object (SWO)

Definition:
A specific physical system, equipment, or sub-process within a Primary Work Object.

Purpose in WRCF:
Enables granular capability and task definition.

Examples (Crushing & Sizing PWO):

Coal Crusher

Crusher Drive System

Crusher Rotor

Vibrating Screen

Screen Drive

Key Insight:
SWO = Where tasks are actually performed.

6 Capabilities

Definition:
Observable, role-agnostic abilities required to effectively interact with a work object.

Capabilities describe *how a person engages with a system*, not what equipment they touch.

Code	Capability
CAP-01	Work Object Familiarization
CAP-02	Process Familiarization (Input → Output)
CAP-03	Operation Execution
CAP-04	Monitoring & Exception Detection
CAP-05	Fault / Error Recognition
CAP-06	First Response & Resolution
CAP-07	Preventive / Proactive Action
CAP-08	Optimization & Judgment
CAP-09	Compliance & Safety Awareness
CAP-10	Documentation, Communication & Handover

Key Insight:
Capability = Type of engagement with system.

Your 10 capabilities are a complete universe, not a checklist.

For any given Secondary Work Object, only a subset is meaningful, observable, and validatable

Example: Conveyor Pull-Cord Switch (SWO)

Which capabilities make sense?

CAP-01 Work Object Familiarization ✔

CAP-04 Monitoring & Exception Detection ✔

CAP-05 Fault Recognition ✔

CAP-09 Compliance & Safety ✔

Which do NOT make sense?

CAP-03 Operation Execution ❌ (you don't "operate" a pull cord)

CAP-08 Optimization & Judgment ❌

CAP-07 Preventive / Proactive Action ❌

CAP-10 Documentation & Handover ❌ (implicit, not primary)

👉 Forcing all 10 would create fake skills and fake control points.

That destroys credibility.

So the rule is:

Each SWO must have only the capabilities that:

result in observable human action
can be tied to a control point
can be validated by a supervisor

Anything else should not be instantiated.

Capability vs Competence

Capability is a reusable human action ability that can be trained and developed independent of role or context while

Competence is the verified ability to apply one or more capabilities effectively, safely, and consistently in a specific role, asset, and process context.

```java
Industry Capability Framework
 ├── Capability (CAP-01, CAP-02, …)
 │     └── Proficiency Levels (L1–L4)
 │           └── Skill Units (atomic tasks)
 │
 └── Role Definition
       └── Required COMPETENCE
             ├── Asset / Process context
             ├── Required Capabilities
             ├── Required Proficiency
             └── Evidence rules
```

So:

- **Capabilities belong to the industry**
- **Competence belongs to the role**
- **People are assessed against competence**

Proficiency

Definition

The degree of autonomy, judgment, and responsibility with which a capability is exercised.

Proficiency Level	Label	Description	Risk Exposure
L1	Plant Awareness	Knowledge only	Zero
L2	Assisted Execution	Perform under observation	Zero
L3	Conditional Independence – Supervised	Perform alone, supervisor nearby	Low
L4	Conditional Independence – Scoped	Perform within boundaries	Controlled
L5	Full Independence	Asset/ Service -level autonomy	Institutionally backed

L1 – Awareness - Validate conceptual understanding before physical execution

System shall assign plant process learning modules linked with respective role in relation to requisite:

Functional Group, PWO, SWO, Capability

The system shall conduct knowledge assessment before L2 unlock.

L2 – Assisted Execution

Demonstrate controlled task execution under supervision. ("Show me you can do it")

During Internship creation, based on the role attached to Internship, User can select predefined Skills and respective tasks while creating Weekly Schedule

Student shall log execution attempt and capture respective evidence against each task. Mentor shall validate performance live or post-task. System shall require X successful supervised attempts before successful completion of task.

Risk:
Supervisor exposure = zero (observational only).

L3 Conditional Independence

Test low-risk autonomy with standby supervision. ("Do it, I'm nearby")

Typically in the initial weeks of Internships Only low-risk tasks are being assigned

Mentor should remain "On Standby" during execution. System shall enforce minimum successful repetition threshold. Safety violation shall auto-reset stage progression.

Risk:
Supervisor risk = low and controlled.

L4– Scoped Independence

Independence is boundary-constrained. ("You can do this — within boundaries")

System shall define operational boundaries per asset:

Example parameters:

Temperature range
Vibration limit
Misalignment tolerance
Pressure threshold

Student shall log independent task performance. The system shall define escalation triggers. System shall record:

Incident-free execution
Correct escalation events
Time-to-escalation
Decision quality

Risk:
Shared between supervisor and system.

L5 - Full Independence - Asset/ Service -level autonomy

Key rule

Proficiency is **how well**, not **what.** It is applied to **capabilities**. It is used to **define trust boundaries**

📌 **Proficiency answers: "How independently can this action be performed?"**

7 Skill

Definition:
A specific, demonstrable ability that enables a person to perform a defined action under a given capability for a given work object.

Structure:
Skill =
(PWO + SWO + Capability + Proficiency)

Example:

"Start crusher drive motor as per SOP"
"Identify belt misalignment condition"
"Monitor dust extraction airflow trend"

Important Clarification:
Skills are operationally meaningful.
Skills are validated through tasks.

```
PWO
 └─ SWO
      └─ Capability (CAP-01 … CAP-10)
           └─ Proficiency (L1–L4)
                └─ Skill (Capability-level)
                     └─ Task (Internship-executable)
                          └─ Evidence
```

🔎 How Everything Connects (Hierarchy View)

```
Industry
   → Functional Group
      → Primary Work Object (PWO)
            → Secondary Work Object (SWO)
            → Capability
              → Proficiency
                  → Skill
                       → Task
                         → Control Point
                                 → Evidence
                               → Evaluator
```

Task

📌 Definition

A **Task** is a clearly defined, observable unit of work performed on a specific Secondary Work Object to demonstrate a specific Skill under a defined Capability and Proficiency level.

It is the **smallest executable field action** that can be:

Assigned during internship
Performed on the shop floor
Observed by a supervisor
Logged in the system
Validated objectively

Structure of a Task

Task = (PWO + SWO + Skill + Context + Mode of Execution)

Example:

"Start crusher drive motor as per SOP and record start-up current"

Control Points

Definition:
Observable validation checkpoints that confirm whether a skill has been correctly demonstrated.

Control Points define:

What must be verified
What evidence is acceptable
Who validates it

Examples:

SOP followed step-by-step
Alarm acknowledged within time limit
Inspection recorded in logbook
Hazard escalated as per protocol

Control Point Structure:
Skill → Task → Control Point → Evidence → Evaluator

Key Insight:
Control Points create trust.

They convert “He knows” into
“He has demonstrated and been validated.”

Tasks build evidence

Evidence validates skills

Validated skills authorize work

Evidence

📌 Definition

Evidence is the documented or observable proof that a Task was executed correctly and that the associated Skill has been demonstrated to the required proficiency level.

Evidence converts:

“I did it” into “It is verifiably done.”

📌 Evidence Types in WRCF

Evidence Type	Description	Example
Observation	Direct visual confirmation	Supervisor observed SOP compliance
Log Entry	Entry in official register	Start-up current logged
Checklist	Signed inspection checklist	Drain inspection record
Alarm Log	Control room record	Alarm acknowledged in DCS
Photo / Video	Visual record	PPE compliance photo
Drill Record	Simulation participation	Fire drill sign-off
Trend Review	Analytical validation	Vibration trend analysis
Incident Report	Structured event documentation	Root cause report
Communication Log	Coordinated action	Shift handover note

📌 Evidence Must Be:

Verifiable
Traceable
Linked to specific task
Linked to evaluator
Time-stamped
Immutable (ideally)

Additional Attributes You Should Store

We want to measure with attributes:

- Skill gaps
- Risk exposure
- Compliance criticality
- Revenue impact
- Downtime sensitivity
- Learning prioritization
- Promotion Readiness

FUNCTIONAL GROUP (FG) — Context Layer

✔ Name

✔ Description

✔ Domain Type (Operations / Maintenance / Quality)

PRIMARY WORK OBJECT (PWO) — Business Impact Layer

✔ Strategic Importance Level (1–5)

✔ Revenue Impact Level (Low / Medium / High)

✔ Downtime Sensitivity (Low / Medium / High)

✔ Dependency Links (optional)

Logic: Revenue + Downtime = Business Priority

SECONDARY WORK OBJECT (SWO) — Operational Risk Layer

✔ Operational Complexity (Low / Medium / High)

✔ Asset Criticality (Low / Medium / High)

✔ Failure Frequency Indicator (Low / Medium / High)

Risk = Complexity + Criticality + Failure likelihood

CAPABILITY INSTANCE

CI = SWO + Capability + Proficiency

No Attributes to be added.it will be computed from

PWO (business impact)

SWO (risk)

Capability (type)

Proficiency (depth)

Compute CI Score

CI Business Impact = PWO Revenue + Downtime

CI Risk Score = SWO Complexity + Criticality + Failure Frequency

CI Criticality Index = f(Impact × Risk × Proficiency)

SKILL — Learning & Decay Layer

✔ Cognitive Type (Procedural / Decision / Diagnostic)

✔ Skill Criticality (Low / Medium / High)

✔ Recertification Cycle (months)

✔ AI Impact (Low / Medium / High)

TASK — Execution Layer

✔ Frequency (Daily / Weekly / Rare)

✔ Complexity (Low / Medium / High)

✔ Standard Duration (optional)

✔ Required Expertise Level

CONTROL POINT — Compliance Layer

✔ Risk Level (Low / Medium / High / Critical)

✔ Failure Impact Type (Safety / Compliance / Financial)

✔ KPI Threshold (optional)

✔ Escalation Required (Yes / No)

DESIGN PRINCIPLES (PIN THESE)

1. **Functional Groups define where work happens.**
2. **Work Objects are the anchor**
3. **Capabilities are universal**
4. **Proficiency defines autonomy**
5. **Skills are atomic actions**
6. **Control points define trust**
7. **Courses prepare; internships validate**
8. **Roles declare Competency**

WRCF Schema

CORE TENANT + VERSION MODEL

```
CREATE TABLE tenants (
  id UUID PRIMARY KEY,
  name TEXT NOT NULL,
  type TEXT CHECK (type IN ('INDUSTRY_MASTER', 'COMPANY')),
  industry_id UUID NULL,
  parent_tenant_id UUID NULL,
  created_at TIMESTAMP DEFAULT now()
);
```

```
CREATE TABLE wrcf_versions (
  id UUID PRIMARY KEY,
  tenant_id UUID REFERENCES tenants(id),
  version_number TEXT,
  is_published BOOLEAN DEFAULT FALSE,
  is_active BOOLEAN DEFAULT TRUE,
  based_on_version_id UUID NULL,
  created_at TIMESTAMP DEFAULT now(),
  published_at TIMESTAMP
);
```

👉 Every entity below links to **version_id + tenant_id**

2️⃣ INDUSTRY STRUCTURE

Functional Group

```
CREATE TABLE functional_groups (
  id UUID PRIMARY KEY,
  tenant_id UUID,
  version_id UUID,
  name TEXT,
  description TEXT,
  domain_type TEXT,
  is_active BOOLEAN DEFAULT TRUE
);
```

Primary Work Object (PWO)

👉 **Business Impact Layer**

```
CREATE TABLE pwo (
  id UUID PRIMARY KEY,
  tenant_id UUID,
  version_id UUID,
  functional_group_id UUID REFERENCES functional_groups(id),

  name TEXT,
  description TEXT,

  strategic_importance SMALLINT,        -- 1–5
  revenue_impact TEXT,                  -- LOW/MEDIUM/HIGH
  downtime_sensitivity TEXT,            -- LOW/MEDIUM/HIGH

  is_active BOOLEAN DEFAULT TRUE
);
```

Secondary Work Object (SWO)

👉 **Operational Risk Layer**

```
CREATE TABLE swo (
  id UUID PRIMARY KEY,
  tenant_id UUID,
  version_id UUID,
  pwo_id UUID REFERENCES pwo(id),

  name TEXT,
  description TEXT,

  operational_complexity TEXT,          -- LOW/MEDIUM/HIGH
  asset_criticality TEXT,               -- LOW/MEDIUM/HIGH
  failure_frequency TEXT,               -- LOW/MEDIUM/HIGH

  is_active BOOLEAN DEFAULT TRUE
);
```

3️⃣ UNIVERSAL TABLES

Capability (GLOBAL)

```
CREATE TABLE capabilities (
  id UUID PRIMARY KEY,
  code TEXT UNIQUE,
  name TEXT,
  type TEXT, -- Cognitive / Execution / Diagnostic
  is_active BOOLEAN DEFAULT TRUE
);
```

Proficiency (GLOBAL)

```
CREATE TABLE proficiencies (
  id UUID PRIMARY KEY,
  level INT,
  label TEXT,
  description TEXT,
  independence_level TEXT
);
```

4️⃣ CAPABILITY INSTANCE (PURE RELATION)

👉 NO attributes here

```
CREATE TABLE capability_instances (
  id UUID PRIMARY KEY,

  tenant_id UUID,
  version_id UUID,

  functional_group_id UUID,
  pwo_id UUID,
  swo_id UUID,
  capability_id UUID,
  proficiency_id UUID,

  UNIQUE (
     tenant_id,
     version_id,
     functional_group_id,
     pwo_id,
     swo_id,
```

```
    capability_id,
    proficiency_id
  )

);
```

5 SKILL LAYER

```
CREATE TABLE skills (
  id UUID PRIMARY KEY,
  tenant_id UUID,
  version_id UUID,

  capability_instance_id UUID REFERENCES capability_instances(id),

  name TEXT,
  description TEXT,

  cognitive_type TEXT,          -- Procedural / Decision / Diagnostic
  skill_criticality TEXT,       -- LOW/MEDIUM/HIGH
  recertification_cycle INT,    -- months
  ai_impact TEXT,               -- LOW/MEDIUM/HIGH

  is_active BOOLEAN DEFAULT TRUE
);
```

6 TASK LAYER

```
CREATE TABLE tasks (
  id UUID PRIMARY KEY,
  tenant_id UUID,
  version_id UUID,

  skill_id UUID REFERENCES skills(id),

  name TEXT,
  description TEXT,

  frequency TEXT,               -- Daily / Weekly / Rare
  complexity TEXT,              -- LOW/MEDIUM/HIGH
  standard_duration INT,        -- minutes
  required_expertise_level INT,

  is_active BOOLEAN DEFAULT TRUE
);
```

7 CONTROL POINT LAYER

```
CREATE TABLE control_points (
  id UUID PRIMARY KEY,
  tenant_id UUID,
  version_id UUID,

  task_id UUID REFERENCES tasks(id),

  name TEXT,
  description TEXT,

  risk_level TEXT,            -- LOW/MEDIUM/HIGH/CRITICAL
  failure_impact TEXT,        -- Safety / Compliance / Financial
  kpi_threshold NUMERIC,
  escalation_required BOOLEAN,

  is_active BOOLEAN DEFAULT TRUE
);
```

8 DEPARTMENT + ROLE

1 **Industry**

Owns Functional Groups

2 **Functional Group**

Defines work domains (Operations, Maintenance, Quality, etc.)

3 **Department**

Operational organizational unit
Can map to multiple Functional Groups

Example:
Maintenance Department
→ Functional Groups: Boilers, Turbines etc

4 Role

Belongs to a Department
Inherits context from Department's Functional Groups
Has Capability Instances attached

Department

```
CREATE TABLE departments (
  id UUID PRIMARY KEY,
  tenant_id UUID,
  version_id UUID,

  name TEXT,
  description TEXT,

  is_active BOOLEAN DEFAULT TRUE
);
```

Department ↔ Functional Group Mapping

```
CREATE TABLE department_function_groups (
  id UUID PRIMARY KEY,
  department_id UUID,
  functional_group_id UUID
);
```

Role

```
CREATE TABLE roles (
  id UUID PRIMARY KEY,
  tenant_id UUID,
  version_id UUID,

  department_id UUID REFERENCES departments(id),

  name TEXT,
  description TEXT,

  is_active BOOLEAN DEFAULT TRUE
);
```

Role ↔ Capability Instance Mapping

```
CREATE TABLE role_capability_instances (
  id UUID PRIMARY KEY,
  role_id UUID REFERENCES roles(id),
  capability_instance_id UUID REFERENCES capability_instances(id),

  mandatory BOOLEAN DEFAULT TRUE,
  weight NUMERIC DEFAULT 1
);
```

9️⃣ DERIVED / COMPUTED LAYER (CRITICAL)

👉 DO NOT STORE — COMPUTE

You will compute:

Example View: Capability Instance Score

```
CREATE VIEW ci_scores AS
SELECT
  ci.id,

  p.revenue_impact,
  p.downtime_sensitivity,

  s.operational_complexity,
  s.asset_criticality,
  s.failure_frequency,

  pr.level AS proficiency_level

FROM capability_instances ci
JOIN pwo p ON ci.pwo_id = p.id
JOIN swo s ON ci.swo_id = s.id
JOIN proficiencies pr ON ci.proficiency_id = pr.id;
```

14. Hybrid Delta Model

1. Company references Industry template
2. Only store company-specific overrides
3. During runtime, merge master + overrides

Example:

Company wants to change:

- failureEscalationLevel
- regulatorySensitivityLevel

So you store:

```
{
 tenantId: CompanyTenantId,
 sourceTemplateId: MasterCapabilityInstanceId,
 isOverride: true,
 overrideFields: [
   "failureEscalationLevel",
   "regulatorySensitivityLevel"
 ],
 failureEscalationLevel: 5,
 regulatorySensitivityLevel: 4

}
```

At runtime:

System loads master
 Then overlays company overrides

This keeps:

✔ 95% reuse
✔ No duplication explosion
✔ Clean upgrade path

Indexing Strategy (Important for Multi-Tenant)

Create compound indexes:

{ tenantId: 1, isActive: 1 }

{ tenantId: 1, sourceTemplateId: 1 }

{ tenantId: 1, industryId: 1 }

All queries must always filter by tenantId.

Never query without a tenantId filter.

You now get:

- Role → Capability Gap
- Department → Capability Heatmap
- Functional Group → Cross-role capability density
- Industry → Exposure analytics

Relationship Flow

Industry
→ FunctionalGroup
→ Department (maps to FGs)
→ Role (belongs to Department)
→ CapabilityInstance
→ Skill → Task → ControlPoint

When I**ndustry updates (publishes) WRCF**:

- Master version increments
- Companies get “Upgrade Available”
- Admin chooses:
    - Accept update
    - Reject
    - Merge selectively

This prevents breaking live certifications.

For **company WRCF,**

🔹 Step 1 — Company Dashboard

Shows:

Industry Version: 4.2
Company Version: 4.2-C1
Overrides: 27

Sections:

- Structural Overrides
- Capability Overrides

- Role Overrides
- Upgrade Available (if any)

🔧 Step 2 — Customize Capability Instance

🏢 Step 3 — Company Role Adjustments

Company can:

- Add new internal roles
- Adjust capability weights
- Change certification threshold

UI shows:

Industry Role Template
Company Role Variant

Color coding:
Green → Same as Industry
Orange → Overridden
Red → Removed

🔁 Step 4 — Upgrade & Merge Flow

When Industry releases Version 5:

Company sees:

Upgrade Available → v5.0

Screen shows diff:

New Capability Instances
Modified Escalation Levels
Updated Regulatory References

Company Options:

- Accept All
- Review Individually
- Defer Upgrade

Review Individually opens side-by-side comparison.

Each change has:
[Accept] [Keep Current]

Final Step:
Generate Company Version 5-C1

🔐 Permissions Layer (Important)

Different UX per user:

Industry Architect:
Full edit access

Company Admin:
Override only

Auditor:
View + export only

Employee:
View role capability requirements only

Context Id:

Our Scenario: (Steel Plant or Cement Plant should be able to use Thermal Power Plant WRCF within Steel plant without duplicating the WRCF). We want to reuse the entire Thermal Power Plant WRCF inside Steel/Cement

Steel Plant

```
Steel Plant
 ├── Blast Furnace System
 ├── Rolling Mill System
 ├── Coke Oven System
 └── Captive Power Plant System  ← (this is key)
      ↳ uses Thermal Power Plant WRCF
```

Think **WRCF is hosted in an Industry but** Industries don't own WRCF — they **use** WRCF. Like lego it should be able to move to the requisite place.

⚠️ Important clarification

❌ so instead of Creating new FG like:

- "Captive CHP FG"
- "Steel CHP FG"

We ARE

✔ Reusing and creating generic names for FG like:

Coal Handling System
Boiler System
Turbine System

🔁 Where Context actually fits

Now context becomes **Qualifier, not connector**

```
System Instance: Captive Power Plant (Steel)
Context:
  systemType: Thermal Power Plant
  subType: Captive
  hostIndustry: Steel
```

We need a System Registry

🔷 System Definition (Reusable)

```
{
  "systemId": "SYS-TPP",
  "systemName": "Thermal Power Plant",
  "wrcfRoot": "FG-Root-TPP"
}
```

🔷 Industry Composition

```
{
  "industry": "Steel",
  "systems": [
    "SYS-BlastFurnace",
    "SYS-RollingMill",
    "SYS-TPP"
  ]
}
```

🔷 System Instance (Context applied)

```
{
  "systemInstanceId": "INST-CPP-Steel",
  "systemId": "SYS-TPP",
  "contextId": "CTX-CPP-Steel"
}
```

👉 That's it.

No pulling.
No duplication.
Just **reference + context**.

So what actually happens in UI / system?

When user selects:

👉 Steel Plant

System shows:

- Steel Systems
- AND
- Thermal Power Plant (as Captive)

When user clicks CHP:

👉 It loads **same WRCF as TPP**

But:

- Labels: "Captive Power Plant"
- Behavior: modified via context

What we MUST do

✔ Define **System = reusable unit**
✔ Attach system to industries
✔ Apply context per instance

Final mental model

- WRCF = library (FG→PWO→SWO→Skill→Task→CP)
- System = module (e.g., Thermal Power Plant)
- Industry = composition of systems
- Context = configuration of a system instance

2. Core Entities (with schemas)

🔷 A. System (Reusable Module)

```
{
  "systemId": "SYS-TPP",
  "systemName": "Thermal Power Plant",
  "description": "Coal-based thermal power generation system",
  "wrcfRootFgIds": ["FG-CHP", "FG-Boiler", "FG-Turbine"],
  "version": "1.0",
  "status": "Active"
}
```

`wrcfRootFgIds` point to your existing FG hierarchy roots.

🔷 B. Industry

```
{
  "industryId": "IND-STEEL",
  "industryName": "Steel",
  "description": "Integrated steel manufacturing"
}
```

🔷 C. Industry-System Mapping (Composition)

```
{
  "industrySystemId": "IS-STEEL-TPP",
  "industryId": "IND-STEEL",
  "systemId": "SYS-TPP",
  "displayName": "Captive Power Plant",
  "isMandatory": true,
  "sequence": 4
}
```

This is where **Steel includes TPP** (no duplication).

🔷 D. Context (Configuration)

```
{
  "contextId": "CTX-CPP-Steel",
  "systemType": "Thermal Power Plant",
  "subType": "Captive",
  "hostIndustry": "Steel",
  "plantScale": "Medium",
  "automationLevel": "Semi-Automated",
  "operatingMode": "Integrated",
  "environmentFactors": ["High Dust", "High Vibration"],
  "criticalityProfile": "Process Critical"
}
```

🔷 E. System Instance (Key Runtime Object)

```
{
  "systemInstanceId": "INST-STEEL-CPP-01",
  "industrySystemId": "IS-STEEL-TPP",
  "contextId": "CTX-CPP-Steel",
  "plantId": "PLANT-001",
  "name": "CPP Unit 1",
  "status": "Active"
}
```

This is what the user actually interacts with.

🔗 3. Context Mapping to WRCF

🔷 F. WRCF Context Mapping

```
{
  "mappingId": "MAP-TPP-CHP-STEEL",
  "systemId": "SYS-TPP",
  "wrcfElementId": "FG-CHP",
  "contextId": "CTX-CPP-Steel",
  "applicability": "Full",
  "variationType": "Modified"
}
```

🔷 G. Context Overrides (Diff Layer)

```
{
  "overrideId": "OVR-001",
  "contextId": "CTX-CPP-Steel",
  "entityType": "Task",
  "entityId": "TASK-Belt-Loading",
  "overrides": {
    "frequency": "Per Shift",
    "complexity": "High",
    "notes": "Higher variability in steel plant"
  }
}
```

Only store changes — never duplicate full data

---

4. Full Data Model (Relational View)

Core WRCF Tables

FG
PWO
SWO
Skill
Task
ControlPoint

System Layer Tables

System
Industry
IndustrySystem
SystemInstance
Context
WRCF_Context_Mapping
WRCF_Context_Override

5. Data Flow (End-to-End)

Step 1: User selects Industry

👉 Steel Plant

Step 2: Load Systems

Steel →

Blast Furnace

Rolling Mill

Captive Power Plant (SYS-TPP)

Step 3: Load System Instance

CPP Unit 1

Context: Steel Captive

Step 4: Load WRCF

SYS-TPP →

FG: Coal Handling

FG: Boiler

FG: Turbine

Step 5: Apply Context

Base Task:

Inspect belt loading → Daily

Context Override:

→ Per Shift

Step 6: Render Final View

User sees:
✔ Same WRCF
✔ Context-adjusted behavior

📊 6. Example (Concrete)

Base WRCF

FG: Coal Handling

PWO: Conveyor

SWO: Belt Operation

Task: Monitor loading (Daily)

Steel Context Output

Task: Monitor loading (Per Shift)

Extra: Check dust accumulation

Cement Context Output

Task: Monitor loading (Per Shift)

Extra: Clean dust deposits

⚠️ 7. Critical Design Rules

1. Systems are reusable

✔ One TPP system
✔ Used everywhere

2. Industry = composition only

✔ No WRCF inside Industry

3. Context = override only

✔ No duplication
✔ Only differences

4. System Instance = execution unit

✔ All assessments tied here

💡 8. API Design (practical)

Get Industry Systems

GET /industries/{id}/systems

Get System Instance WRCF

GET /system-instances/{id}/wrcf

Apply Context (backend logic)

load base WRCF

apply context overrides

return final structure

🚀 9. UI Structure

Industry View

- Steel Plant
    - Blast Furnace
    - Rolling Mill
    - Captive Power Plant

System View

- Coal Handling
- Boiler
- Turbine

Task View

- Shows:
  - Task
  - Control Points
  - Evidence
  - Evaluator

🔥 10. What we've built (big picture)

With this system:

✔ One WRCF → multiple industries
✔ No duplication
✔ Context-driven behavior
✔ Scalable to any domain